

React Hooks 最佳实践与常见陷阱

useState Hook:

- 状态更新是异步的，连续调用 setState 不会立即更新
- 函数式更新确保基于最新状态：setCount(prev => prev + 1)
- 复杂状态考虑使用 useReducer

useEffect Hook:

常见陷阱:

1. 缺少依赖项导致闭包问题

// 错误示例

```
useEffect(() => {  
  console.log(count); // 总是初始值  
}, []);
```

// 正确做法

```
useEffect(() => {  
  console.log(count);  
}, [count]);
```

2. 无限循环

```
useEffect(() => {  
  setCount(count + 1); // 每次渲染都触发  
}, [count]); // 导致无限循环
```

3. 清理函数缺失

```
useEffect(() => {
```

```
const subscription = api.subscribe();

return () => subscription.unsubscribe(); // 必须清理

}, []);
```

useMemo 和 useCallback:

- useMemo 缓存计算结果，避免重复计算
- useCallback 缓存函数引用，避免子组件不必要重渲染
- 不要过度优化，只在必要时使用

自定义 Hook:

- 以"use"开头，方便 ESLint 识别
- 可组合多个内置 Hook
- 示例：useLocalStorage、useFetch

性能优化:

1. React.memo 包裹组件，避免不必要渲染
2. 使用 useMemo 缓存昂贵计算
3. 代码分割：React.lazy + Suspense
4. 虚拟列表处理大数据

测试策略:

- 使用@testing-library/react
- 模拟 Hook: jest.mock()
- 测试异步效果: act()

常见错误模式:

1. 在条件或循环中调用 Hook

2. 在 class 组件中使用 Hook
3. 忘记处理 loading 和 error 状态
4. useEffect 依赖项过多